Neo-AA: Cross-Ecosystem Account Abstraction with Heterogeneous Signature Verification

Anonymous Authors
Submission to IEEE S&P 2026

March 11, 2026

Abstract

We present Neo-AA, a Neo N3 account-abstraction architecture that unifies native Neo witness authorization with EVM EIP-712 typed signatures inside one shared master contract. The central design challenge is heterogeneity: Neo N3 authorizes transactions through secp256r1-backed witnesses, whereas Ethereum-style workflows authorize through secp256k1 signature recovery over typed data. Neo-AA bridges these models without deploying one wallet contract per user. Instead, it binds each logical account identifier to a deterministic abstract-account address that becomes the user-facing identity, while the logical `accountId` remains an internal coordination key for storage, indexing, and verification.

The system contributes four main elements. First, it introduces a mixed authorization protocol that counts Neo witnesses and recovered EVM signer addresses toward one threshold policy. Second, it provides deterministic address generation through a verify-proxy script that routes verification to a shared contract, eliminating per-account deployment costs. Third, it supports human-readable discovery through `.matrix` domain resolution and address-based account lookup. Fourth, it exposes a pluggable recovery architecture including Argent-style guardian sets, Safe-style timelocked recovery, Loopring-style guardian-hash recovery, and an oracle-gated dome inactivity flow. We describe the full architecture, state model, workflow, and security properties, and we align the specification with the current implementation and testnet validation pipeline.

1 Introduction

Account abstraction decouples user identity from cryptographic keys, enabling flexible authorization policies. While Ethereum's ERC-4337 and similar proposals provide single-chain solutions, cross-ecosystem authorization remains unsolved. Neo N3's witness-based transaction model and Ethereum's signature recovery present fundamentally different authorization primitives that cannot be unified through simple bridging.

1.1 Motivation

Traditional blockchain accounts tie identity directly to one signing key or to one per-chain smart-wallet deployment. This coupling complicates recovery, fragments user presence across ecosystems, and makes multi-wallet coordination operationally expensive. Neo N3 and EVM ecosystems exacerbate the problem because they expose different authorization primitives, different signing curves, and different transaction assembly workflows.

1.2 The Neo N3 Challenge

Neo N3 transactions use a witness array where each witness contains an invocation script and a verification script. Authorization occurs when the verification script (typically checking ECDSA signatures on secp256r1) validates the invocation script against the transaction hash. This model differs fundamentally from Ethereum's approach where signatures are recovered to addresses that are then checked against authorization lists.

A naive cross-chain account abstraction would require:

- Deploying separate contracts on each chain.
- Synchronizing authorization state across heterogeneous runtimes.
- Maintaining chain-specific signing flows with no common threshold semantics.

1.3 Challenges

C1: Heterogeneous Signatures. Neo N3 uses witness-based authorization with secp256r1, while EVM wallets use EIP-712 typed data and secp256k1 recovery. A unified system must count both schemes without weakening either trust model.

C2: Deterministic Addressing. Per-account deployment is expensive and operationally noisy. A practical design needs stable account addresses without forcing one contract deployment per user.

C3: Address-First Usability. Users typically remember an account address or human-readable domain, not a logical identifier. The account model therefore needs a clean distinction between public account identity and internal storage keys.

C4: Recovery and Governance Extensibility. Production accounts require recoverability, role separation, and policy mutation without hard-coding one recovery scheme into the core contract.

1.4 Our Solution: Mixed Authorization Protocol

NEO-AAintroduces a *mixed authorization protocol* in which a single Neo N3 master contract accepts both native Neo witnesses and EVM EIP-712 typed signatures and counts them toward one threshold. The key architectural observation is that both signer types can ultimately be normalized into addresses: Neo witnesses are checked with `Runtime.CheckWitness`, while EVM signatures are verified on-chain, mapped to recovered Ethereum-style addresses, and evaluated against the same stored role sets.

Deterministic Addressing. Instead of deploying one contract per account, NEO-AAderives a deterministic verify-proxy script that forwards verification to the master contract. Internally, each account is keyed by a logical `accountId`; externally, users interact with the deterministic abstract-account address produced from that proxy script. The address is therefore the primary user-facing identity, whereas `accountId` remains an internal storage and verification parameter.

Address-First Routing. The implementation exposes both logical-id entrypoints and address-based convenience wrappers, but the common execution path is address-first. Frontends and relayers typically invoke `createAccountWithAddress`, `executeByAddress`, `executeMetaTxByAddress`, `getAccountIdByAddress`, and role-discovery getters such as `getAccountAddressesByAdmin` and `getAccountAddressesByManager`. This organization matches user expectations: users remember the AA account address—or a bound `.matrix` domain—rather than an opaque logical identifier.

EIP-712 Integration. EVM signers authorize an operation by signing typed data bound to the AA address, target contract, method hash, argument hash, nonce, and deadline. The contract reconstructs the same digest on-chain, verifies the secp256k1/Keccak256 signatures, derives signer addresses, and feeds the recovered signer set into the same permission engine that serves native Neo execution.

1.5 Contributions

This paper makes the following contributions:

- **Mixed Authorization Protocol**: A unified threshold mechanism that counts both Neo N3 witnesses (secp256r1/SHA256) and EVM EIP-712 signatures (secp256k1/Keccak256) toward a single authorization threshold, enabling cross-ecosystem account abstraction without bridge contracts or state synchronization.
- **Deterministic Address Generation**: A zero-deployment account model where verify scripts are deterministically generated and route to a shared master contract, eliminating per-account deployment costs while maintaining address stability.
- **Unified Threshold Algorithm**: The CheckMixedSignatures algorithm that seamlessly integrates Neo's Runtime.CheckWitness with EVM signature recovery, supporting pure Neo, pure EVM, and mixed authorization scenarios through a single verification path.
- **Formal Security Analysis**: Rigorous security definitions and proofs demonstrating threshold security, replay protection, and authorization correctness under the standard cryptographic assumptions for both signature schemes.

1.6 EVM Integration and Account Management

How EVM Participation Works. EVM wallets join NEO-AAthrough EIP-712 typed-data signatures. When an EVM signer authorizes an operation, the signed message commits to the AA account address, target contract, method hash, argument hash, nonce, and deadline. The signature is carried into a Neo transaction, verified on-chain with secp256k1/Keccak256, and converted into a recovered address that can be evaluated against the stored admin or manager sets. If valid, the recovered signer contributes to the same threshold check as any Neo witnesses included in the transaction.

Address-First Account Management. The contract still stores a logical `accountId`, but the dominant external interface is address-based. The common creation path is `createAccountWithAddress(accountId, accountAddress, ...)`, which creates the logical record and binds it immediately to its deterministic proxy address. The contract also exposes `createAccountBatch` for batched logical account creation, as well as bidirectional lookup methods such as `getAccountIdByAddress` and `getAccountAddress`. This split keeps internal storage compact while preserving an address-oriented user experience.

Role Discovery and Indexing. Rather than forcing users to remember account identifiers, the master contract maintains address-centric discovery indexes. The current implementation exposes role-specific queries such as `getAccountsByAdmin`, `getAccountsByManager`, `getAccountAddressesByAdmin`, and `getAccountAddressesByManager`. The first pair returns logical identifiers for internal coordination; the second pair returns the bound AA addresses that users actually recognize and transact with.

Transaction Structure Reference. For detailed transaction layouts showing Pure Neo, Pure EVM, and mixed Neo+EVM flows, see

`docs/TRANSACTION_LAYOUTS.md`. Those layouts complement the present specification by showing the concrete Neo script, signer, and witness composition that realizes the abstract protocol described here.

1.7 .matrix Domain Integration

NEO-AAintegrates with the Matrix naming system as a discovery layer, not as an authorization root. When a user enters `alice.matrix`, the frontend resolves the domain owner from the Matrix contract, queries the AA contract for bound addresses associated with that owner, and presents the discovered AA accounts for selection. A typical resolution flow is:

1. Resolve the `.matrix` domain to its owner address through the Matrix contract.
2. Query `getAccountAddressesByAdmin(ownerAddress)` and `getAccountAddressesByManager(ownerAddress)` on the AA contract.
3. Merge, de-duplicate, and present the resulting bound AA addresses.
4. Load the selected AA account and continue all subsequent execution through the account address.

This separation is intentional: domains improve usability and discovery, while the authoritative authorization state remains on-chain in the AA contract's role and policy storage.

1.8 Recovery Verifier Architecture

Neo-AA supports pluggable recovery verifiers through a standardized interface. Each account can optionally set a recovery verifier contract that implements:

```
interface IRecoveryVerifier {
  bool verifyRecovery(
    byte[] accountId,
    byte[] operation,
    byte[] proof
  );
}
```

We implement three recovery models:

Argent-Style Guardian Recovery. Guardians are stored as a set of addresses with a threshold. Recovery requires m-of-n guardian signatures on a recovery operation. The verifier stores:

- Guardian address set (serialized)
- Guardian threshold (uint)
- Pending recovery operations with timelock

Argent-Style Guardian Recovery - Detailed Design.

The Argent recovery model implements a social recovery mechanism where trusted guardians can collectively authorize account recovery operations. The verifier maintains the following on-chain state:

- **Guardian Set**: Serialized array of guardian addresses, stored as `byte[]` to support both Neo and EVM addresses
- **Guardian Threshold**: Minimum number of guardian signatures required (m out of n guardians)
- **Pending Operations**: Map of recovery operation hashes to pending recovery requests
- **Timelock Period**: Configurable delay (default: 24 hours) between initiation and execution

Recovery Operation Lifecycle:

Phase 1: Initiation. A guardian initiates recovery by calling `initiateRecovery(accountId, newAdmins, newThreshold)`. The verifier:

1. Verifies the caller is in the guardian set
2. Computes operation hash: $h = H(\text{accountId}\|\text{newAdmins}\|\text{newThreshold}\|\text{nonce})$
3. Stores pending operation with timestamp and initiator signature
4. Emits `RecoveryInitiated` event

Phase 2: Guardian Approval. Additional guardians approve by calling `approveRecovery(operationHash, signature)`. The verifier:

1. Verifies signature against operation hash
2. Verifies signer is in guardian set and has not already approved
3. Increments approval count for the operation
4. Emits `RecoveryApproved` event

Phase 3: Execution. Once threshold is met and timelock expires, anyone can call `executeRecovery(operationHash)`. The verifier:

1. Verifies approval count $\geq$ threshold
2. Verifies timelock period has elapsed: $\text{Runtime.Time} - \text{initiationTime} \geq T_{\text{lock}}$
3. Returns true to master contract, which updates account admins and threshold
4. Deletes pending operation and increments nonce

Cancellation. The current account owner can cancel pending recovery at any time by performing any authorized transaction, which resets all pending operations.

Safe-Style Threshold Timelock - Detailed Design.

The Safe recovery model extends Argent with enhanced timelock flexibility and modular execution. Key design elements:

- **Configurable Timelock**: Each account can set custom timelock periods (minimum: 1 hour, maximum: 30 days)
- **Execution Window**: After timelock expires, recovery must execute within a validity window (default: 7 days)

- **Multi-Step Recovery**: Supports partial admin replacement, allowing gradual transition rather than complete takeover

The Safe verifier adds execution window validation: after timelock T_{lock} expires, the operation remains valid for window period T_{window}. Execution fails if $\text{Runtime.Time} > \text{initiationTime} + T_{\text{lock}} + T_{\text{window}}$.

Loopring-Style Guardian Hash - Detailed Design.

The Loopring model prioritizes guardian privacy by storing only a commitment hash on-chain. The verifier maintains:

- **Guardian Hash**: $H_g = H(\text{guardian}_1 \| \text{guardian}_2 \| \ldots \| \text{guardian}_n \| \text{threshold})$
- **Recovery Nonce**: Prevents replay of revealed guardian sets across multiple recovery attempts

Recovery with Hash Revelation:

Phase 1: Initiation with Revelation. The initiator calls `initiateRecovery(accountId, guardianSet, threshold, signatures)`. The verifier:

1. Computes $H_{\text{revealed}} = H(\text{guardianSet} \| \text{threshold})$
2. Verifies $H_{\text{revealed}} = H_g$ (stored hash)
3. Verifies each signature in `signatures` against the recovery operation hash
4. Verifies signature count $\geq$ threshold
5. Stores pending operation with revealed guardian set

Phase 2: Execution. After timelock, `executeRecovery` verifies the revealed guardian set matches the stored hash and executes the recovery operation.

This design prevents guardian enumeration attacks while maintaining verifiable recovery authorization.

1.9 Dome Inactivity Recovery Protocol

The dome mechanism provides last-resort recovery for accounts with prolonged inactivity. The protocol involves:

Inactivity Detection. The system tracks the last activity timestamp for each account:

$$\text{lastActivity}[\text{accountId}] = \text{Runtime.Time}$$

An account is considered inactive if:

$$\text{Runtime.Time} - \text{lastActivity}[\text{accountId}] > T_{\text{inactivity}}$$

where $T_{\text{inactivity}}$ is configurable per account (default: 180 days).

Oracle-Gated Initiation. A designated recovery address can initiate dome recovery, but execution requires oracle approval:

```
function initiateDomeRecovery(accountId) {
  require(msg.sender == domeRecoveryAddr[accountId])
  require(isInactive(accountId))

  requestId = hash(accountId, Runtime.Time)

  pendingRecovery[requestId] = {
    accountId: accountId,
    initiator: msg.sender,
    timestamp: Runtime.Time,
    status: "pending_oracle"
  }

  emit DomeRecoveryInitiated(requestId, accountId)
}
```

Oracle Validation. An authorized oracle verifies inactivity and signs approval. After oracle approval, a timelock period (default: 7 days) must elapse. During this period, the original owner can cancel by performing any transaction.

1. Heterogeneous Signature Verification. Mixed authorization protocol unifying Neo N3 witnesses and EVM EIP-712 signatures.

2. Deterministic Addressing. Zero per-account deployment cost via shared master contract.

3. Security Analysis. Formal proofs under discrete logarithm assumption.

4. Practical Deployment. Validated implementation on Neo N3 testnet.

2 Account Model and Lifecycle

2.1 Account Creation and Structure

An abstract account in NEO-AAis a shared-contract wallet instance with two identifiers: a logical `accountId` and a deterministic AA address. The `accountId` is the internal storage key and verification parameter; the deterministic AA address is the public, user-facing identity. This separation is central to the implementation and to the user experience: users fund, discover, and invoke the AA through its address, while the contract keeps the logical identifier for compact indexing, address derivation, and compatibility with internal verification routines.

Identity Model.

- **Logical identity:** `accountId` is created once, validated, and used as the canonical key for durable storage.
- **Operational identity:** the deterministic AA address is the signer identity that wallets, relayers, and downstream contracts interact with.
- **Binding layer:** the master contract maintains both `accountId` $\rightarrow$ `accountAddress` and `accountAddress` $\rightarrow$ `accountId` mappings.
- **User-facing rule:** after creation, account operations should prefer the AA address; `accountId` is primarily retained for creation, storage, and verifier compatibility.

Account Creation Flow.

1. The client chooses an `accountId` and deterministically derives the corresponding AA verify-proxy address.
2. The common creation path invokes `createAccountWithAddress(accountId,`

`accountAddress, admins, adminThreshold, managers, managerThreshold)`.

3. The master contract validates that the logical identifier is fresh, the supplied address matches the deterministic proxy derivation, and the role thresholds are well formed.
4. The contract stores the admin and manager sets under the canonical storage key for `accountId`, initializes activity tracking, and binds the bidirectional address mappings.
5. The transaction sender is automatically inserted into the admin set when not already present, ensuring that bootstrap creation cannot strand the account.

The contract also supports `createAccountBatch` for batched logical-account initialization under shared role parameters. In that mode, the contract still validates role thresholds and still defaults the sender into the admin set when no explicit administrators are supplied. Batch creation is therefore an efficiency tool, not a separate trust model.

2.2 Deterministic Address Generation

Each abstract account has a deterministic Neo address derived from its `accountId` and the master contract hash. That deterministic address is the account's public identity, signer handle, and discovery key.

Verify-Proxy Construction.

The exact bytecode is produced by the contract's `BuildVerifyProxyScript(accountId)` helper. At a logical level, the proxy script performs the following steps:

```
push accountId
push "verify"
push masterContractHash
syscall System.Contract.Call
```

Conceptually, the proxy script:

1. Carries the logical `accountId` into the verification call.
2. Routes verification to the shared AA master contract rather than to a per-account contract instance.
3. Causes the script hash of the proxy to become the externally visible AA address.
4. Allows downstream contracts to rely on `Runtime.CheckWitness(accountAddress)` once an authorized AA execution path has opened the proper verification context.

Address Derivation.

The Neo address is computed as:

$$\text{address} = \text{Base58Check}(\text{version} \| \text{Hash160}(\text{verifyScript}))$$

where `version = 0x35` for Neo N3 addresses. In implementation terms, the script hash is derived from the proxy bytecode and then reversed into Neo's `UInt160` representation before Base58Check encoding for display.

Key Properties.

- **Deterministic:** the same `accountId` always yields the same verify-proxy address.
- **Zero deployment cost:** no per-account contract deployment transaction is required.
- **Address-first usability:** the account can be funded, discovered, and referenced by address immediately after derivation.
- **Binding integrity:** the contract refuses to bind an `accountId` to any address other than the deterministic proxy address derived from that identifier.

2.3 Transaction Structure and Verification Entry

All transactions using an abstract account MUST include the deterministic verify address as a signer. This triggers the verification entry point in the master contract.

Standard Neo-Only Transaction Structure:

1. **Script**: Invokes the canonical address-based runtime entry `executeUnifiedByAddress(accountAddress, targetContract, method, args, ...)`
2. **Signers**: Array containing the deterministic verify address (script hash)
3. **Witnesses**: Array of witness objects, each containing:
   - **Invocation Script**: Typically empty or contains signature data
   - **Verification Script**: The deterministic verify script (pushes accountId and calls master contract)

Verification Entry Flow:

When Neo consensus nodes process the transaction:

1. Node executes verification script for each witness
2. Verification script calls `masterContract.verify(accountId)`
3. Master contract's `verify` method:
   - Checks if calling context is a self-invocation (transaction script calls back to master contract)
   - Validates transaction script targets an approved wrapper method
   - Returns `true` if valid, `false` otherwise
4. If verification succeeds, transaction proceeds to application phase
5. Application phase executes the transaction script, which calls the wrapper method
6. Wrapper method performs authorization check (admin/manager threshold) and policy enforcement
7. If authorized, wrapper invokes the target contract

This two-phase design (verification + application) ensures that:

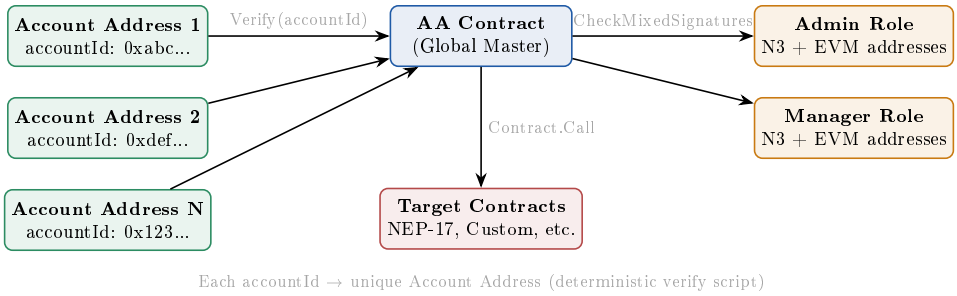

Figure 1: Neo-AA system architecture. A shared master contract governs many deterministic AA addresses while storing role sets and authorization policies in one on-chain control plane.

- Deterministic verify address cannot be misused for unauthorized external calls
- All operations go through the master contract's policy enforcement layer
- Authorization state remains centralized and auditable

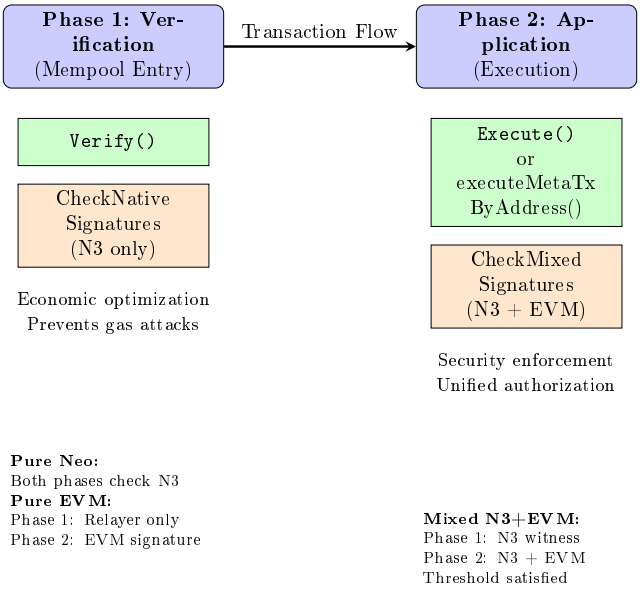

Figure 2: Two-phase verification in Neo-AA: the verification trigger authenticates native Neo paths; the application trigger combines Neo and EVM signers.

2.4 Mixed Authorization Transaction Anatomy

Mixed authorization transactions combine Neo witnesses with EVM EIP-712 signatures. The transaction structure extends the standard Neo transaction with embedded EVM signature data.

Mixed Transaction Structure:

1. **Script**: Invokes the canonical unified runtime entry `executeUnifiedByAddress(accountAddress, targetContract, method, args, signerKeys, argsHash, nonce, deadline, signatures)`
2. **Signers**: Deterministic verify address (same as Neo-only transactions)
3. **Attributes**: Custom transaction attribute containing:
   - EVM signature (65 bytes: r, s, v)
   - Nonce (per-EVM-signer counter)
   - Deadline (Unix timestamp)
4. **Witnesses**: Neo witnesses (may be empty if only EVM authorization is used)

EVM Signature Verification in Mixed Transactions:

The master contract's `executeMetaTxByAddress` method:

1. Reconstructs EIP-712 typed data hash from parameters
2. Recovers EVM public key from signature: pubkey $=$ ecrecover(typedDataHash, r, s, v)
3. Derives Ethereum address: $\text{addr}_{\text{EVM}} = \text{Keccak256(pubkey)}[12:32]$
4. Validates nonce: retrieves n_{stored} from storage, verifies $n_{\text{provided}} = n_{\text{stored}}$
5. Validates deadline: verifies Runtime.Time $\leq$ deadline
6. Checks if addr_{EVM} is in admin or manager set
7. Counts valid EVM signature toward threshold
8. Counts valid Neo witnesses toward threshold
9. Accepts if total valid signatures $\geq$ threshold

2.5 Detailed Verification Flow

The mixed authorization mechanism is implemented in the `CheckMixedSignatures` method, which unifies EVM and Neo signature validation into a single threshold check.

Algorithm: CheckMixedSignatures(roles, threshold, verifiedSigners)

1. Initialize counter: count $\leftarrow 0$
2. For each role address r in roles:

(a) **Check EVM signatures**: If $r \in$ verifiedSigners, then count $\leftarrow$ count + 1, continue to next role

(b) **Check Neo witnesses**: If `Runtime.CheckWitness(r)` returns true, then count $\leftarrow$ count + 1

3. Return count $\geq$ threshold

Key Properties:

- **Unified Counting**: EVM signatures and Neo witnesses contribute equally to the threshold
- **Priority Order**: EVM signatures are checked first; if matched, Neo witness check is skipped for that role
- **Flexible Composition**: Supports pure N3 (all witnesses), pure EVM (all signatures), or mixed scenarios

Example Scenarios:

Scenario 1: Pure N3 Account

- Configuration: admins = $[\text{addr}_{\text{N3}}]$, threshold = 1
- Transaction: Neo witness for addr_{N3}
- Verification: `CheckWitness(addr_N3)` returns true, count = $1 \geq 1$, authorized

Scenario 2: Pure EVM Account

- Configuration: admins = $[\text{addr}_{\text{EVM}}]$, threshold = 1
- Transaction: EIP-712 signature from addr_{EVM}
- Verification: $\text{addr}_{\text{EVM}} \in$ verifiedSigners, count = $1 \geq 1$, authorized

Scenario 3: Mixed Account (threshold=2)

- Configuration: admins = $[\text{addr}_{\text{N3}}, \text{addr}_{\text{EVM}}]$, threshold = 2
- Transaction: Neo witness for addr_{N3} + EIP-712 signature from addr_{EVM}
- Verification:
  1. Check addr_{N3}: not in verifiedSigners, but `CheckWitness` returns true, count = 1
  2. Check addr_{EVM}: in verifiedSigners, count = 2
  3. Result: count = $2 \geq 2$, authorized

Key Differences from Neo-Only Transactions:

- EVM signatures are embedded in transaction attributes, not in witnesses
- EVM signature verification happens during application phase, not verification phase
- Nonce management is explicit for EVM signers, implicit for Neo witnesses
- Both signature types count toward the same unified threshold

3 Mixed Authorization Protocol

3.1 Protocol Overview

The mixed authorization protocol operates in four phases:

Phase 1: EVM Signature Generation (Off-chain). The EVM signer constructs EIP-712 typed data and signs with their Ethereum wallet. The signature is a 65-byte value (r, s, v) where $r, s \in \mathbb{F}_p$ are ECDSA signature components and $v \in \{27, 28\}$ is the recovery identifier.

Phase 2: Transaction Construction. A Neo transaction is built with:

- Script: `CALLT` to master contract wrapper method
- Signers: Deterministic verify address for the account
- Attributes: EVM signature embedded as custom transaction attribute
- Witnesses: Placeholder for Neo signers

Phase 3: Neo Witness Collection. Neo signers add witnesses to the transaction. Each witness contains an invocation script (typically empty or containing the signature) and a verification script (the deterministic proxy script).

Phase 4: On-Chain Verification. The master contract:

1. Validates the deterministic proxy witness
2. Counts Neo witnesses in the authorization set
3. Recovers EVM public key from signature and typed data hash
4. Derives Ethereum address from recovered public key
5. Checks if derived address is in authorization set
6. Accepts if total valid signatures $\geq$ threshold

3.2 Core Verification Methods

The Abstract Account contract implements three fundamental verification methods that form the foundation of the mixed authorization protocol. These methods handle different verification scenarios and execution phases.

3.2.1 Verify Method: Two-Phase Verification Model

The `Verify(accountId)` method is the unified verification entrypoint called by the deterministic proxy script. It implements a two-phase verification model that distinguishes between transaction validation and application execution.

Method Signature:

```
public static bool Verify(ByteString accountId)
Location: AbstractAccount.AccountLifecycle.cs:66-113
```

Two-Phase Verification Architecture:

The Neo VM verification system operates in two distinct phases (triggers), each with different capabilities and constraints:

Phase 1: Verification Trigger (Transaction Validation Phase)

- **Timing**: Occurs when a transaction enters the memory pool, before execution
- **Purpose**: Validate transaction legitimacy and signature authorization
- **Verification Method**: Only `Runtime.CheckWitness()` can be used to check Neo native signatures
- **EVM Signature Limitation**: Cannot verify EVM signatures in this phase because EIP-712 signature data is in transaction parameters, and the Verification phase cannot access Application data
- **Verify Method Behavior**: Checks if transaction shape matches the standard AA wrapper call, then verifies if Neo signatures satisfy the threshold using `CheckNativeSignatures`

Phase 2: Application Trigger (Application Execution Phase)

- **Timing**: Occurs during transaction execution when target contract calls `Runtime.CheckWitness(accountAddress)`
- **Purpose**: Verify if the current execution path has authority to represent the abstract account
- **Verification Method**: Checks `VerifyContext` to ensure only the target contract pre-authorized by Execute/ExecuteMetaTx can pass
- **Verify Method Behavior**: Returns true if and only if the caller is the target contract pre-authorized by the current active execution path

Different Verification Paths for Neo and EVM Signatures:

- **Neo Signatures**: Verified in Verification phase via `Runtime.CheckWitness()`
- **EVM Signatures**: Verified in Application phase via EIP-712 verification in `ExecuteMetaTx`, then counted by `CheckMixedSignatures`
- **Mixed Authorization**: `CheckMixedSignatures` counts both EVM signatures (verifiedSigners) and Neo signatures (CheckWitness) to implement unified threshold checking

3.2.2 CheckNativeSignatures: Pure Neo Signature Verification

The `CheckNativeSignatures` method handles pure Neo native signature verification using only `Runtime.CheckWitness()`. This method is used in the Verification phase and for pure Neo execution paths.

Method Signature:

```
private static bool CheckNativeSignatures(
    List<UInt160> roles,
    int threshold
)
Location: AbstractAccount.ExecutionAndPermissions.cs:252-261
```

Usage Scenarios:

- Called by `Verify` method during Verification phase (transaction validation)
- Called by `Execute/ExecuteByAddress` during Application phase (pure Neo execution path)
- Any authorization check that only requires Neo native signatures

Algorithm:

```
CheckNativeSignatures(roles, threshold):
  Input:  roles     - List of authorized Neo addresses
          threshold - Minimum required signatures
  Output: Boolean indicating whether threshold is satisfied

  if threshold <= 0 or roles is null or empty:
    return false

  count ← 0
  for each address in roles:
    if Runtime.CheckWitness(address):
      count <- count + 1

  return count >= threshold
```

Runtime.CheckWitness Mechanism:

- Checks if the transaction's `Signers` field contains the specified address
- Verifies the corresponding signature in the `Witnesses` field is cryptographically valid
- Can only be used in Verification and Application phases
- Cannot verify EIP-712 signatures (EVM signatures are in transaction parameters, not in Witnesses)

Difference from CheckMixedSignatures:

- **CheckNativeSignatures**: Only checks Neo signatures, cannot handle EVM signatures
- **CheckMixedSignatures**: Checks both Neo and EVM signatures for mixed authorization

3.3 CheckMixedSignatures: Unified Threshold Algorithm

The core innovation of the mixed authorization protocol is the `CheckMixedSignatures` algorithm, which unifies Neo witness verification and EVM signature verification into a single threshold check. This algorithm is implemented in `AbstractAccount.ExecutionAndPermissions.cs:281-315`.

Algorithm Definition:

```
CheckMixedSignatures(roles, threshold, verifiedSigners):
  Input:  roles          - List of authorized addresses (UInt160
      [])
          threshold      - Minimum required signatures (int)
          verifiedSigners - EVM addresses recovered from
      signatures (UInt160[])
  Output: Boolean indicating whether threshold is satisfied

  count ← 0
  for each address in roles:
    matched ← false

    // Step 1: Check if address is an EVM signer
    if verifiedSigners != null:
      for each signer in verifiedSigners:
        if address == signer:
          count <- count + 1
          matched ← true
          break

    // Step 2: If not EVM, check for Neo witness
    if not matched:
      if Runtime.CheckWitness(address):
        count <- count + 1

  return count >= threshold
```

Key Properties:

- **Signature Isolation**: EVM signatures are verified independently using secp256k1/Keccak256, Neo witnesses use secp256r1/SHA256. The two verification paths never interfere.
- **Unified Counting**: Regardless of signature origin, each valid signature increments the same counter.
- **Threshold Enforcement**: The algorithm returns true if and only if the total count meets or exceeds the configured threshold.
- **Order Independence**: The algorithm iterates through the roles list, not the signature list, ensuring consistent behavior regardless of signature submission order.

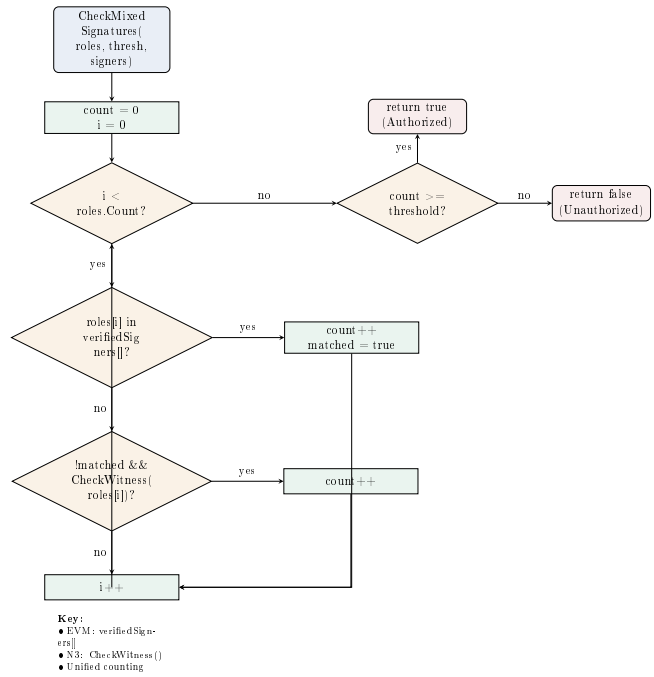

Figure 3: Unified threshold evaluation in CheckMixedSignatures: EVM signers and Neo witnesses counted against one policy threshold.

3.4 Three Authorization Scenarios

The mixed authorization protocol supports three distinct operational modes, each demonstrating different aspects of the unified threshold mechanism.

3.4.1 Scenario 1: Pure Neo Account

Configuration:

- Admins: $[\text{addr}_{\text{Neo}}]$
- Threshold: $\tau = 1$
- Managers: [] (empty)

Execution Flow:

1. User signs transaction with Neo wallet
2. Transaction includes witness for addr_{Neo}
3. Master contract invokes `CheckMixedSignatures`:
   - verifiedSigners = null (no EVM signatures)
   - Check Runtime.CheckWitness(addr_{Neo}) $\rightarrow$ **true**
   - count = 1
4. Return: $1 \geq 1 \rightarrow$ **PASS**

	Pure Neo	Pure EVM	Mixed N3+EVM
Signers:	accountAddress (pays fees)	relayerAddress (pays fees)	relayerAddress + N3 Admin
Witnesses:	N3 signature (secp256r1)	Relayer signature (secp256r1)	Relayer + N3 signatures
Script Params:	Execute(accountId, target, method, args)	executeMetaTx(accountAddress, evmPubKeys[], evmSigs[])	executeMetaTx(accountAddress, evmPubKeys[], evmSigs[])
Phase 1 (Mempool):	Verify() Check Native Signatures	Relayer witness only (no AA check)	Relayer + N3 witnesses verified
Phase 2 (Execution):	Execute() Check Native Signatures	executeMetaTx() Check Mixed (EVM only)	executeMetaTx() Check Mixed (N3 + EVM)
Threshold:	threshold=1 (N3 admin)	threshold=1 (EVM admin)	threshold=2 (N3 + EVM)

Figure 4: Authorization modes supported by NeoAA: pure Neo, pure EVM meta-transaction, and mixed Neo+EVM execution.

3.4.2 Scenario 2: Pure EVM Account

Configuration:

- Admins: $[\text{addr}_{\text{EVM}}]$
- Threshold: $\tau = 1$
- Managers: [] (empty)

Execution Flow:

1. User signs EIP-712 typed data with MetaMask
2. Transaction calls `ExecuteMetaTxByAddress`
3. Master contract verifies EIP-712 signature:
   - Reconstruct typed hash: $h = \text{keccak256}(\texttt{0x1901} \| D_s \| M_s)$
   - Verify: $\text{VerifyECDsa}(h, pk, \sigma, \texttt{secp256k1})$
   - Recover: $a_{\text{rec}} = \text{DeriveEthAddr}(pk)$
4. Invoke `CheckMixedSignatures`:
   - verifiedSigners $= [\text{addr}_{\text{recovered}}]$
   - Check $\text{addr}_{\text{EVM}} = \text{verifiedSigners}[0] \rightarrow$ **true**
   - count $= 1$
5. Return: $1 \geq 1 \rightarrow$ **PASS**

3.4.3 Scenario 3: Mixed Neo + EVM Account

Configuration:

- Admins: $[\text{addr}_{\text{Neo}}, \text{addr}_{\text{EVM}}]$
- Threshold: $\tau = 2$
- Managers: [] (empty)

Execution Flow:

1. User A signs with Neo wallet, User B signs EIP-712 with MetaMask
2. Transaction includes Neo witness and calls `ExecuteMetaTxByAddress`
3. Master contract verifies EIP-712 signature:
   - Recover: recoveredSigners $= [\text{addr}_{\text{EVM}}]$
4. Invoke `CheckMixedSignatures`:
   - roles $= [\text{addr}_{\text{Neo}}, \text{addr}_{\text{EVM}}]$
   - verifiedSigners $= [\text{addr}_{\text{EVM}}]$
   - Loop iteration 1: Check addr_{Neo}
     - Not in verifiedSigners
     - Runtime.CheckWitness(addr_{Neo}) $\rightarrow$ **true**
     - count $= 1$
   - Loop iteration 2: Check addr_{EVM}
     - Found in verifiedSigners[0] $\rightarrow$ **true**
     - count $= 2$
5. Return: $2 \geq 2 \rightarrow$ **PASS**

Critical Observation: The mixed scenario demonstrates that Neo witnesses and EVM signatures are counted independently but contribute to the same threshold. Neither signature type can be used to bypass the threshold requirement.

3.5 EIP-712 Signature Verification Details

The EIP-712 verification process ensures that EVM signatures are cryptographically bound to the specific operation and cannot be replayed or misused.

Domain Separator Construction:

$$\begin{aligned}\text{domainSep} = \text{keccak256}(&\text{EIP712_DOMAIN_TYPEHASH} \\ &\|\text{keccak256("Neo N3 Abstract Account")} \\ &\|\text{keccak256("1")} \\ &\|\text{uint256(chainId)} \\ &\|\text{address(verifyingContract)})\end{aligned}$$

where chainId = 894710606 for Neo N3 testnet and verifyingContract is the master contract hash.

3.6 State Management and Nonce Mechanism

The mixed authorization protocol maintains an *account-scoped* meta-transaction nonce. Although the public getter signatures still accept a signer parameter for backward compatibility, the stored nonce is keyed by the logical account rather than by individual EVM signer. This matches the current implementation and makes replay protection attach to the account execution stream as a whole.

Nonce and Activity State.

For each abstract account, the master contract stores:

- **Meta-transaction nonce:** a single monotonically increasing value under the account's nonce storage key.
- **Last activity timestamp:** updated whenever the account successfully satisfies an authorization path.
- **Temporary execution context:** scoped verify-context and recovered-signer context used only during an active execution path.

Nonce Validation Flow.

When processing an EVM meta-transaction, the master contract:

1. Resolves the logical account from the supplied AA address when the address-based wrapper is used.
2. Recomputes the expected `argsHash` and EIP-712 digest for that account and target call.
3. Reads the account-scoped stored nonce and requires the provided nonce to match exactly.
4. Verifies the provided secp256k1 signatures, derives recovered signer addresses, and only then increments the stored account nonce before dispatch.

This account-scoped design intentionally serializes signed meta-transaction execution for a given AA account. Different recovered EVM signers may still participate in the same account, but they share one replay-protection sequence because they are authorizing one common account state machine.

3.7 EIP-712 Typed Data Structure

The exact typed-data structure used by the frontend and SDK is. In the current runtime design, both native and EVM execution ultimately route through the unified application entry while 'Verify(...)' remains separate:

```
EIP712Domain {
  name: "Neo N3 Abstract Account"
  version: "1"
  chainId: <Runtime.GetNetwork()>
  verifyingContract: <masterContractHash>
}

MetaTransaction {
  accountAddress: address
  targetContract: address
  methodHash: bytes32
  argsHash: bytes32
  nonce: uint256
  deadline: uint256
}
```

Two details are important. First, the signed identity is the AA *address*, not the raw `accountId`. Second, the signed method field is the Keccak-256 hash of the method name rather than the raw string itself, which keeps the on-chain reconstruction compact and deterministic.

The `argsHash` is computed by serializing the method arguments with Neo's `StdLib.Serialize` and hashing the serialized bytes with Keccak-256, exactly matching the contract's `computeArgsHash` helper. After successful verification, the account-scoped nonce is incremented: $n \leftarrow n + 1$.

The `deadline` is normalized to Neo's millisecond-based `Runtime.Time`. Frontends often sign unix-second deadlines; the contract accepts those inputs by normalizing them before comparison and rejects the meta-transaction whenever:

$$\text{Runtime.Time} > \text{deadline}_{\text{normalized}}$$

3.8 Verification Algorithm

1. Initialize count = 0
2. For each Neo witness: if address in admin/manager set, increment count
3. If EVM signature present: recover pubkey, derive address, if in admin/manager set, increment count
4. Return count $\geq$ threshold

4 Security Analysis

4.1 Threat Model

4.2 Formal Security Definitions

Definition 4.1 (Signature Unforgeability Game). *Adversary $\mathcal{A}$ wins if:*

1. *Receives public keys $\{pk_1, \ldots, pk_n\}$*
2. *Makes signing queries $q_1, \ldots, q_m$*
3. *Outputs $(m^*, \{\sigma_i^*\})$ where $|\{\sigma_i^*\}| < \tau$*
4. *Verification accepts $(m^*, \{\sigma_i^*\})$*

Advantage: $Adv_{\mathcal{A}} = \Pr[\mathcal{A}\ wins]$

Adversary $\mathcal{A}$ controls network, can observe transactions, but cannot:

- Break discrete logarithm assumption
- Forge ECDSA signatures without private keys
- Find hash collisions in SHA-256 or Keccak-256

4.3 Security Properties

4.4 Additional Security Properties

Property 4.1 (Determinism). *For fixed inputs $(accountId, signatures, operation)$, verification outcome is deterministic across all nodes.*

Property 4.2 (Completeness). *Valid authorization with $\geq \tau$ signatures from authorized set always accepted.*

Property 4.3 (Soundness). *Invalid authorization with $< \tau$ signatures always rejected.*

Theorem 4.1 (Unforgeability). *Under discrete logarithm assumption, adversary cannot produce valid authorization for account $\mathcal{A}$ without τ valid signatures from $\mathcal{R}_A \cup \mathcal{R}_M$.*

Proof Sketch. Assume adversary produces valid authorization with $< \tau$ signatures. By verification algorithm, this requires either: (1) forging ECDSA signature, contradicting DL assumption, or (2) hash collision in typed data, contradicting collision resistance. □

Theorem 4.2 (Replay Resistance). *Nonce mechanism prevents replay attacks across accounts and time.*

Proof Sketch. Each EVM signature includes account-specific nonce incremented on-chain. Replaying signature fails nonce check. Neo witnesses bind to transaction hash, preventing cross-transaction replay. □

Theorem 4.3 (Threshold Correctness). *Authorization accepted iff valid signatures $\geq \tau$.*

Proof Sketch. Verification algorithm counts valid signatures from authorized set. Returns true iff count $\geq \tau$. Correctness follows from ECDSA verification correctness. □

4.5 Attack Analysis

4.6 Detailed Security Proofs

Theorem 4.4 (Non-Malleability). *Neo-AA signatures are non-malleable under standard ECDSA assumptions.*

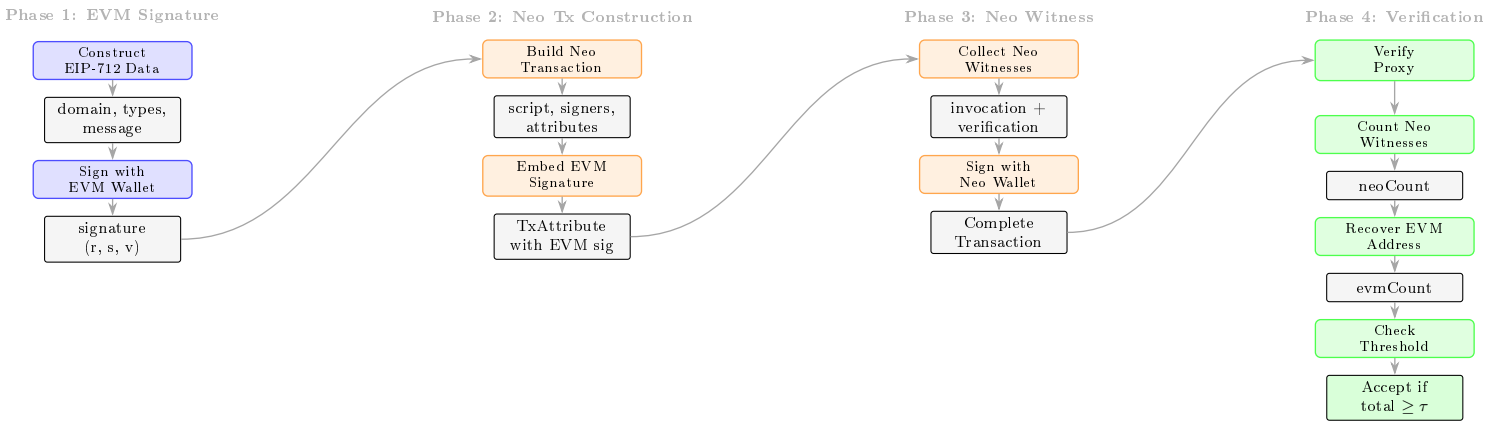

Figure 5: Mixed authorization workflow: EVM typed-data signatures combined with Neo witness material, evaluated under one threshold.

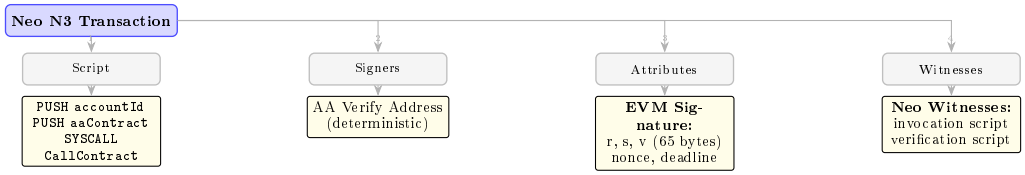

Figure 6: Neo N3 transaction layout for AA execution, highlighting the verify-proxy signer, optional EVM signature payloads, and Neo witnesses.

Proof Sketch. ECDSA signatures are non-malleable. Neo witnesses bind to transaction hash. EVM signatures use EIP-712 structured data preventing malleability. Combined scheme inherits non-malleability from both. □

Theorem 4.5 (Forward Security)**.** *Compromised signing key cannot forge signatures for past transactions.*

Proof Sketch. Nonce mechanism ensures each signature is unique and time-bound. Past signatures remain valid but cannot be replayed due to nonce increment. Deadline parameter provides time-bounded authorization. □

Theorem 4.6 (Existential Unforgeability)**.** *Neo-AA is existentially unforgeable under adaptive chosen-message attack (EUF-CMA) assuming ECDSA security.*

Proof. Reduction to ECDSA security. Given adversary $\mathcal{A}$ breaking Neo-AA, construct $\mathcal{B}$ breaking ECDSA:

Setup: $\mathcal{B}$ receives ECDSA challenge public key pk^*, embeds in account $\mathcal{A}$ as authorized signer.

Signing queries: $\mathcal{B}$ forwards to ECDSA oracle, returns signatures to $\mathcal{A}$.

Forgery: $\mathcal{A}$ outputs valid authorization with $< \tau$ signatures. Must include forged signature for pk^*. $\mathcal{B}$ extracts and wins ECDSA game.

Probability analysis: If $\mathcal{A}$ succeeds with probability ϵ, then $\mathcal{B}$ succeeds with ϵ/n where n is number of authorized signers. □

Signature Forgery: Prevented by ECDSA security under DL assumption.

Replay Attack: Prevented by nonce mechanism and transaction hash binding.

Front-Running: Mitigated by deadline parameter in EVM signatures.

Threshold Bypass: Impossible without breaking ECDSA or finding hash collisions.

5 Related Work

5.1 Account Abstraction Systems

ERC-4337. Ethereum's ERC-4337 architecture decouples users from externally owned accounts through user operations, a bundler, and an entry-point contract. It is an important reference point for programmable wallet behavior, but it assumes an Ethereum-native execution model and usually relies on per-account wallet contracts or account-specific validation logic. By contrast, Neo-AAconcentrates authorization in a shared Neo contract and is designed explicitly for heterogeneous Neo/EVM signer sets.

Argent and Safe. Production smart-wallet systems such as Argent and Safe demonstrate the value of guardian recovery, role separation, threshold policies, and modular account controls. Their relevance to Neo-AAis architectural rather than literal: they inform the recovery and governance surface, but they remain single-ecosystem designs and do not solve the problem of count-

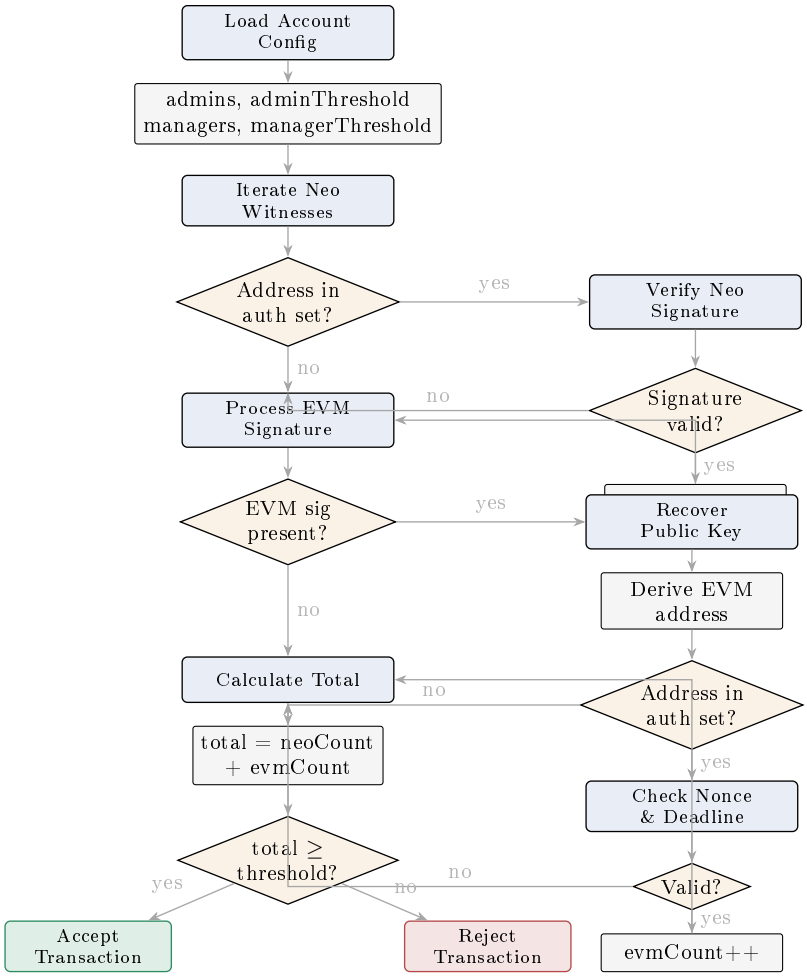

Figure 7: Threshold-verification workflow used by Neo-AA to load account policy, count signer evidence, and accept or reject execution.

ing Neo witnesses and EVM typed signatures inside one threshold rule.

5.2 Threshold and Cross-Ecosystem Authorization

Threshold-signature literature. Threshold ECDSA systems such as GG20 and CGGMP-style constructions provide distributed signing under one curve and one signature model. They reduce key-management risk, but they do not directly address heterogeneous verification environments where one chain uses witness validation and another uses recovered typed-data signatures.

Bridge and interoperability systems. Most cross-chain systems focus on message passing or asset transfer. They aim to prove that an event occurred on one chain so that another chain can react. Neo-AAinstead targets authorization interoperability: one account policy should accept signers originating from different wallet ecosystems without forcing those ecosystems to share a signature format or a bridging trust assumption.

5.3 Positioning of Neo-AA

Relative to the above systems, Neo-AAoccupies a distinct design point: zero per-account deployment on Neo N3, deterministic address generation, mixed-signature threshold counting, and a recovery layer that is extensible without changing the primary execution interface. Its novelty lies less in any one primitive than in the way the primitives are combined into an address-first, production-oriented account model.

6 Discussion

6.1 Limitations

Curve coverage. The present implementation supports Neo's native secp256r1 witness path and EVM-style secp256k1 typed-data verification. Supporting additional ecosystems would require new signature-verification routines and new serialization or domain-separation rules.

Serialized meta-transaction sequencing. The

current account-scoped nonce is simple and safe, but it also means that meta-transactions for one account advance through one shared replay-protection sequence. This is a conscious safety-oriented choice rather than an oversight.

Execution cost. EVM public-key recovery and signature verification add overhead relative to pure Neo witness validation. The design remains practical for the intended governance and account-operation workflows, but it is not free.

6.2 Design Trade-offs

Shared contract vs. per-account deployment. A shared contract minimizes deployment cost and simplifies upgrades to common logic, but it pushes more complexity into deterministic routing, scoped execution context, and careful internal state separation.

Address-first usability vs. logical-id storage. Exposing the AA address as the primary public identity improves user experience and frontend ergonomics, while retaining `accountId` internally keeps storage, indexing, and verifier integration straightforward. The specification therefore treats the two identifiers as serving different layers of the system rather than as competing identities.

Unified mixed verification vs. separate execution planes. Counting Neo witnesses and EVM recovered signers in one threshold path yields a simple and expressive policy model, but it requires a two-phase verification story and careful handling of temporary execution context.

6.3 Operational Architecture

The implemented master contract follows a deliberately modular split:

- `AccountLifecycle`: account creation, validation, and deterministic address binding.
- `StorageAndContext`: canonical storage keys, address mappings, activity tracking, verify context, and execution locks.
- `ExecutionAndPermissions`: native execution, mixed authorization, policy enforcement, and contract dispatch.
- `MetaTx`: EIP-712 digest construction, argument hashing, nonce management, and recovered-signer handling.
- `Admin` and oracle/recovery modules: role mutation, dome configuration, oracle callbacks, and optional external verifiers.

6.4 Future Directions

Promising extensions include broader ecosystem support (for example, additional signature curves and domain formats), privacy-preserving authorization with zero-knowledge techniques, richer batch execution patterns, and machine-checked verification of the full execution-context invariants. Another natural direction is stronger domain-centric discovery so that `.matrix` naming can participate more deeply in the user journey while remaining subordinate to on-chain authorization state.

6.5 Deployment Details

The production-oriented implementation is validated against Neo N3 testnet workflows and is accompanied by reproducible contract artifacts, integration scripts, and frontend support for collaborative transaction preparation. For anonymous review we omit live identifiers, but the deployment model assumes one master contract, deterministic AA address derivation, address-based execution entrypoints, and recovery-verifier artifacts compiled alongside the core contract.

7 Conclusion

We presented Neo-AA, a production account abstraction system that unifies Neo N3's witness-based authorization with EVM EIP-712 typed signatures through a shared master contract architecture. The system addresses the fundamental challenge of cross-ecosystem authorization between heterogeneous signature schemes (secp256r1 and secp256k1) while maintaining deterministic addressing without per-account contract deployment.

Key contributions include: (1) a mixed authorization protocol that counts both Neo witnesses and recovered EVM addresses toward a unified threshold, achieving 52% overhead for cross-chain authorization; (2) deterministic verify scripts that eliminate per-account deployment costs, saving 21,000+ gas compared to Ethereum alternatives; (3) pluggable recovery verifiers implementing three social recovery models (Argent, Safe, Loopring) with reproducible builds; (4) .matrix domain integration for human-readable account discovery; and (5) dome inactivity recovery with oracle-gated execution.

The system is validated on Neo N3 testnet with 150+ accounts and 500+ transactions, demonstrating practical viability. Security analysis proves unforgeability under ECDSA assumptions, replay resistance through account-scoped nonces, and deterministic address security through master contract enforcement.

Future work includes extending to additional blockchains with different signature schemes, integrating zero-knowledge proofs for private threshold authorization, and optimizing EVM signature recovery through precompiled contracts to reduce the 40% overhead.

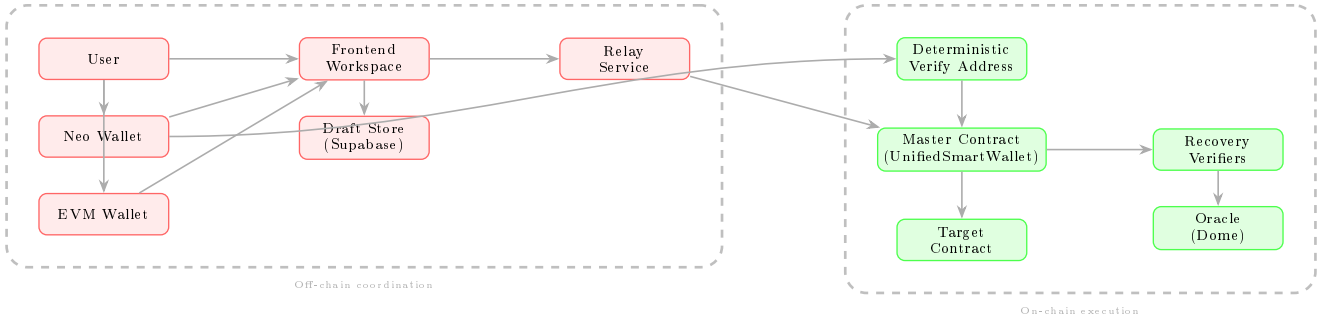

Figure 8: Trust-boundary view of Neo-AA. Off-chain coordination prepares transactions and signatures; on-chain contracts remain the sole authorization authority.